

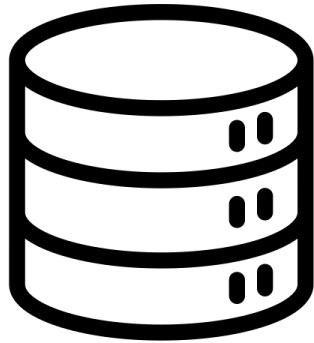
Procesamiento de Datos Masivos

Clase 05 - Data Warehousing II

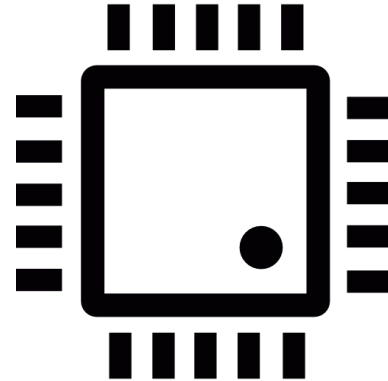
Qué haremos hoy

- Haremos un repaso express de lo que vimos la semana pasada
- Veremos algunos métodos nuevos que podemos usar en BigQuery
- Haremos un ejemplo para comparar una OLTP vs OLAP
- Actividad

Recursos limitados en computación



Almacenamiento

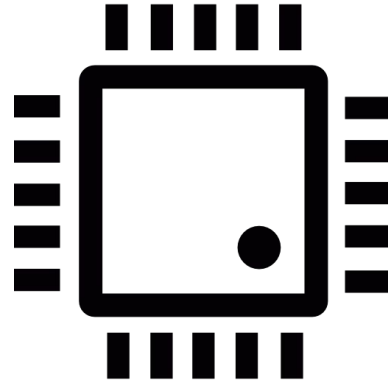


Tiempo

Recursos limitados en computación



Almacenamiento



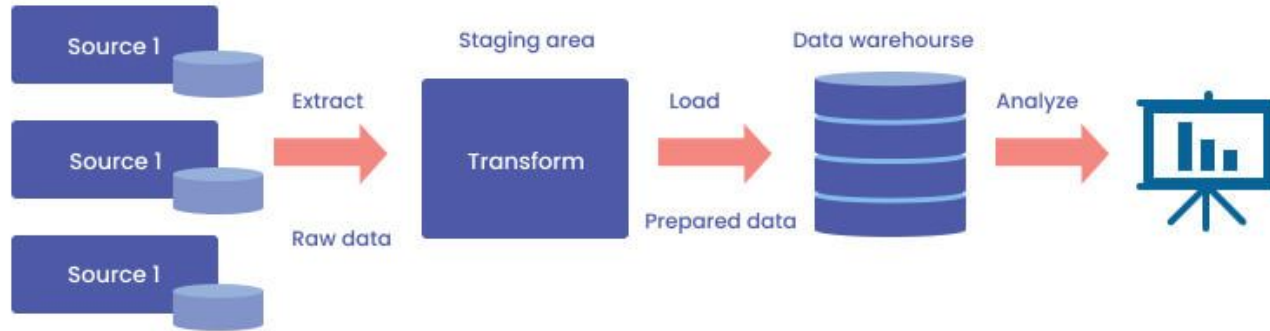
Tiempo

Objetivo del capítulo

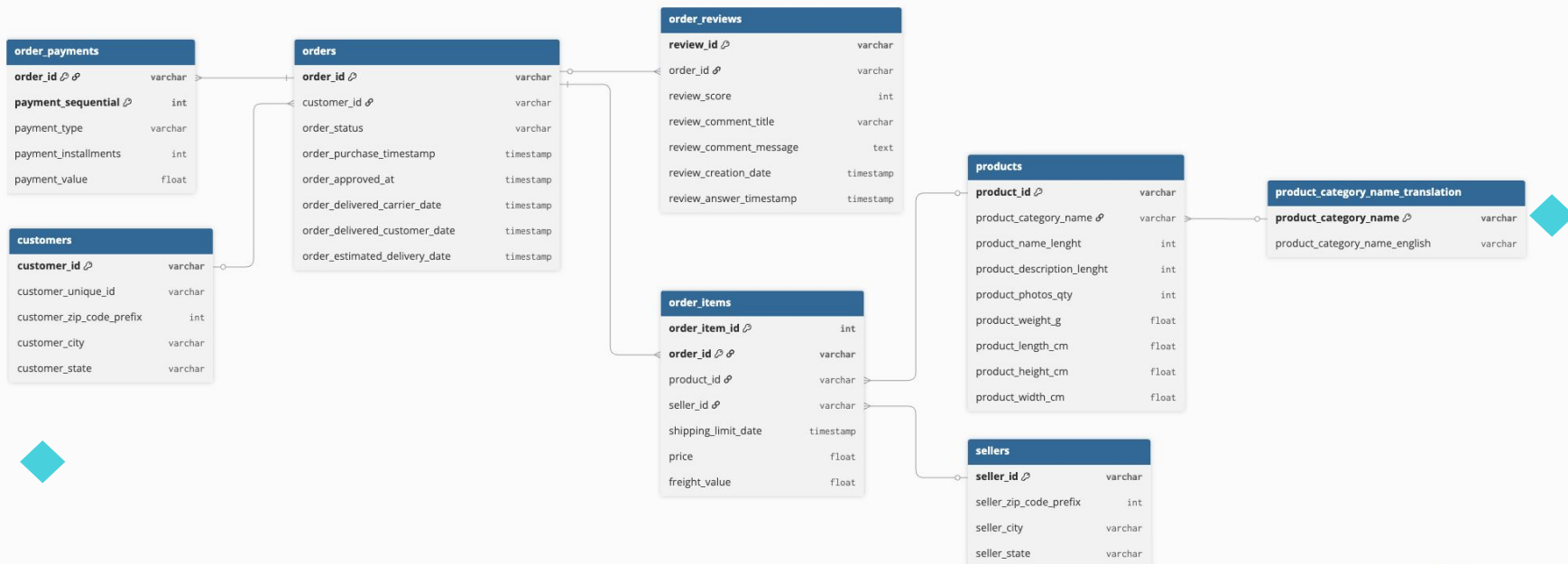
Nos gustaría hacer reportería analizando todos nuestros datos y así responder a preguntas como:

- Cuáles fueron las top 3 productos más vendidos por mes
- Cual es la diferencia de ventas de un producto por trimestre
- Ventas acumuladas por mes
- Relaciones entre nuestros datos y fuentes externas

Objetivo del capítulo



DB en producción



1



Consultando en producción

Teóricamente es posible, pero

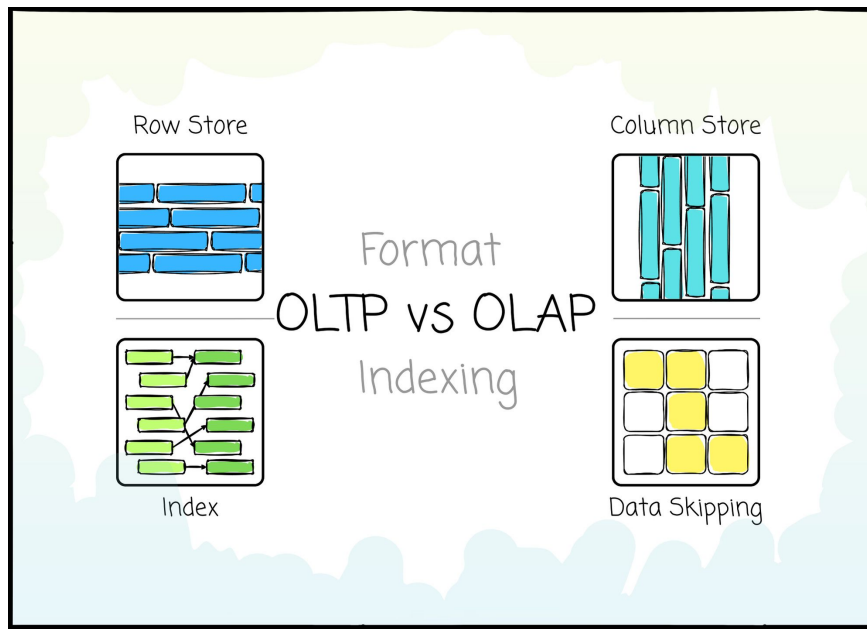
- Consultas complejas tomarán tiempo
- Modelamiento no optimizado para grandes lecturas
- Bloqueo de tablas y uso de recursos afectarán al negocio
- Difícil realizar cruces con otras fuentes de datos

OLAP



Un warehouse

- Nos da un motor optimizado para consultas analíticas
- Permite centralizar información limpia y procesada usando modelos más eficientes para la lectura
- Entorno para no afectar a usuarios



Recordemos

- Las páginas de disco las leemos por columna
- No tenemos índices
- Aprovechamos **particionamiento** y **clustering** para optimizar el acceso a los datos

BigQuery

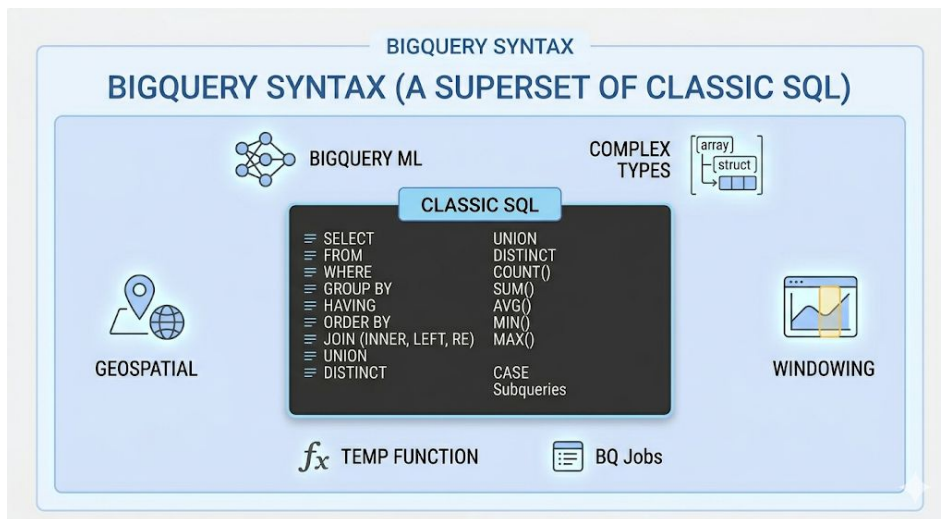


BigQuery

- BigQuery es una DB columnar serverless diseñada para analítica mediante cómputo distribuido
- Optimizada para agregaciones y consultas masivas
- Flujo de una query
 1. Envías una query SQL
 2. BigQuery la divide en tareas
 3. Se asignan múltiples nodos de cómputo, donde cada nodo
 - a. Lee solo las columnas necesarias
 - b. Procesa una parte de los datos
 4. Se combinan los resultados

Sintaxis BigQuery

- La sintaxis de BigQuery extiende a la de SQL



Ejemplo: dónde se queda chico SQL

- Tenemos los videos de una plataforma
- Para cada video, queremos obtener el mes con más vistas por año
- Intento 1:

```
SELECT name, year, MAX(views)
FROM videos
GROUP BY name, year
```

* Supongan que *views* son el total de vistas por mes
** Estamos usando [estos datos](#)


```
query = """
SELECT name, year, MAX(views)
FROM youtube_videos_dataset
GROUP BY name, year
"""

headers = ["name", "year", "max_views"]
print(tabulate(run_query(query), headers=headers, tablefmt="grid"))
```

✓ 0.0s

name	year	max_views
A method expect son those.	2022	1291636
Perhaps away.	2022	1769176
Maintain project follow.	2022	1777448
For pull never.	2022	1758355

- Tenemos los videos de una plataforma
- Para cada video, queremos obtener el mes con más vistas por año
- Intento 2:

```
SELECT *  
FROM videos v  
JOIN (  
    SELECT name, year, MAX(views) as max_views  
    FROM videos  
    GROUP BY name, year  
) t  
ON v.name = t.name AND v.year = t.year AND v.views = t.max_views
```

```

query = """
SELECT t.name, t.year, v.month, v.views
FROM youtube_videos_dataset v
JOIN (
    SELECT name, year, MAX(views) as max_views
    FROM youtube_videos_dataset
    GROUP BY name, year
) t
ON v.name = t.name AND v.year = t.year AND v.views = t.max_views
"""

headers = ["name", "year", "month", "max_views"]
print(tabulate(run_query(query), headers=headers, tablefmt="grid"))

```

✓ 0.0s

name	year	month	max_views
Thousand institution.	2022	4	1896357
Measure gun crime power game.	2022	5	1597631
Early board.	2022	1	1907901
Force tax trouble product give.	2022	8	1780310

- Tenemos los videos de una plataforma
- Para cada video, queremos obtener el mes con más vistas por año
- SQL clásico no nos permite hacer esto de forma simple ni eficiente

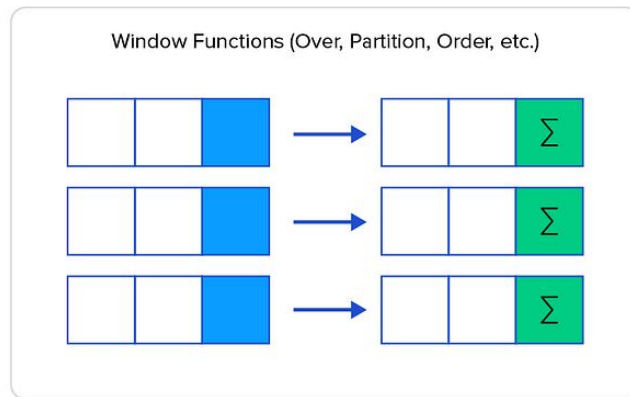
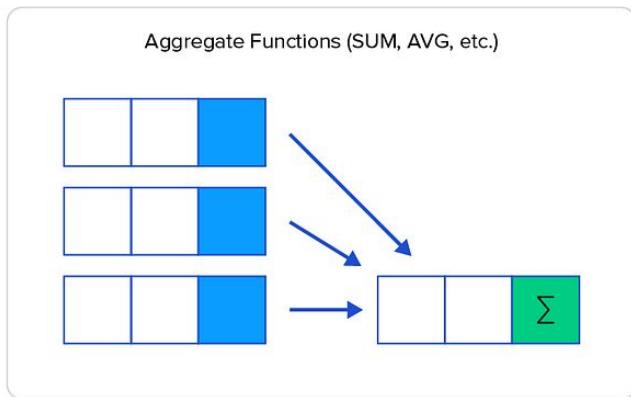
- Tenemos los videos de una plataforma
- Para cada video, queremos obtener el mes con más vistas por año
- SQL clásico no nos permite hacer esto de forma simple ni eficiente
- Uno de los principales problemas es que tenemos que **colapsar** la información para llegar a resultados

- Tenemos los videos de una plataforma
- Para cada video, queremos obtener el mes con más vistas por año
- SQL clásico no nos permite hacer esto de forma simple ni eficiente
- Uno de los principales problemas es que tenemos que **colapsar** la información para llegar a resultados
- Aquí las **Window Functions** vienen a desplegar su fuerza

Window Functions

Las Window Functions permiten hacer analítica compleja sin mover ni colapsar los datos

Podemos calcular valores sobre grupos de filas **manteniendo cada fila original**



Window Functions

- Trabajamos sobre “ventanas” de datos, definidas por:
 - **PARTITION BY**
 - **ORDER BY**
 - **FRAME**
- Cada partición puede ejecutarse en nodos distintos
- **Minimiza shuffles** innecesarios (comunicación)
- **PARTITION BY** no es solo semántico: define **cómo se paraleliza** el problema

PARTITION BY

- “Quiero calcular algo por grupo sin hacer **GROUP BY**”

```
SELECT *,  
SUM(views) OVER (PARTITION BY name) AS total_views  
FROM videos
```

- A diferencia de **GROUP BY**, esto nos permite mantener todas las filas y añadir este total como una columna nueva

ROW_NUMBER(), RANK(), DENSE_RANK()

- “Quiero el top N dentro de cada grupo”

```
SELECT *,  
ROW_NUMBER() OVER (PARTITION BY name, year ORDER BY views DESC) AS ranking  
FROM videos
```

- Estos operadores nos permiten ordenar dentro de un grupo y asignar un número a este orden
- row_number son valores únicos por grupo (1, 2, ...) y no hay empates. rank permite empates pero deja vacíos (1, 2, 2, 4, ...). dense_rank elimina estos vacíos (1, 2, 2, 3, ...)

QUALIFY

- “Quiero filtrar después de calcular ranking o métricas”

```
SELECT *  
FROM videos  
QUALIFY ROW_NUMBER() OVER (PARTITION BY name, year ORDER BY views DESC) <= 3
```

- Con esto, filtramos resultados de window functions sin la necesidad de subqueries

Slide (ventanas temporales)

- “Quiero calcular la media móvil”

```
SELECT *,  
  AVG(views) OVER (  
    PARTITION BY name  
    ORDER BY month  
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW  
  ) AS moving_avg  
FROM videos
```

- Cuidado cuando falten meses para un dato y los borde de un slide

Con BigQuery

- Tenemos los videos de una plataforma
- **Para cada video, queremos obtener el mes con más vistas por año**

Con BigQuery

- Tenemos los videos de una plataforma
- Para cada video, queremos obtener el mes con más vistas por año

```
1 SELECT *,
2   ROW_NUMBER() OVER (PARTITION BY name, year ORDER BY views DESC) AS ranking
3 FROM datosmasivos-2025.videos.videos
4 QUALIFY ranking=1
5
```

✓ Esta consulta procesará 7.56 KB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

[+ Crear conversación](#)

[Guardar los resultados](#)

[Abrir en](#)

Información del trabajo

Resultados

Visualización

JSON

Detalles de la ejecución

Gráfico de ejecución

Fila	name	category	year	month	views	likes	ranking
1	A method expect son those.	Música	2022	11	1291636	50117	1
2	Different life who.	Deportes	2022	8	1171332	85121	1

Resumiendo

- BigQuery extiende la sintaxis de SQL
- Esta nos permite hacer consultas sin perder información por colapsar los datos
- Operadores como **PARTITION BY** no solo son una forma de trabajar los datos, sino que nos indica cómo distribuir los nodos de procesamiento

Ejemplos

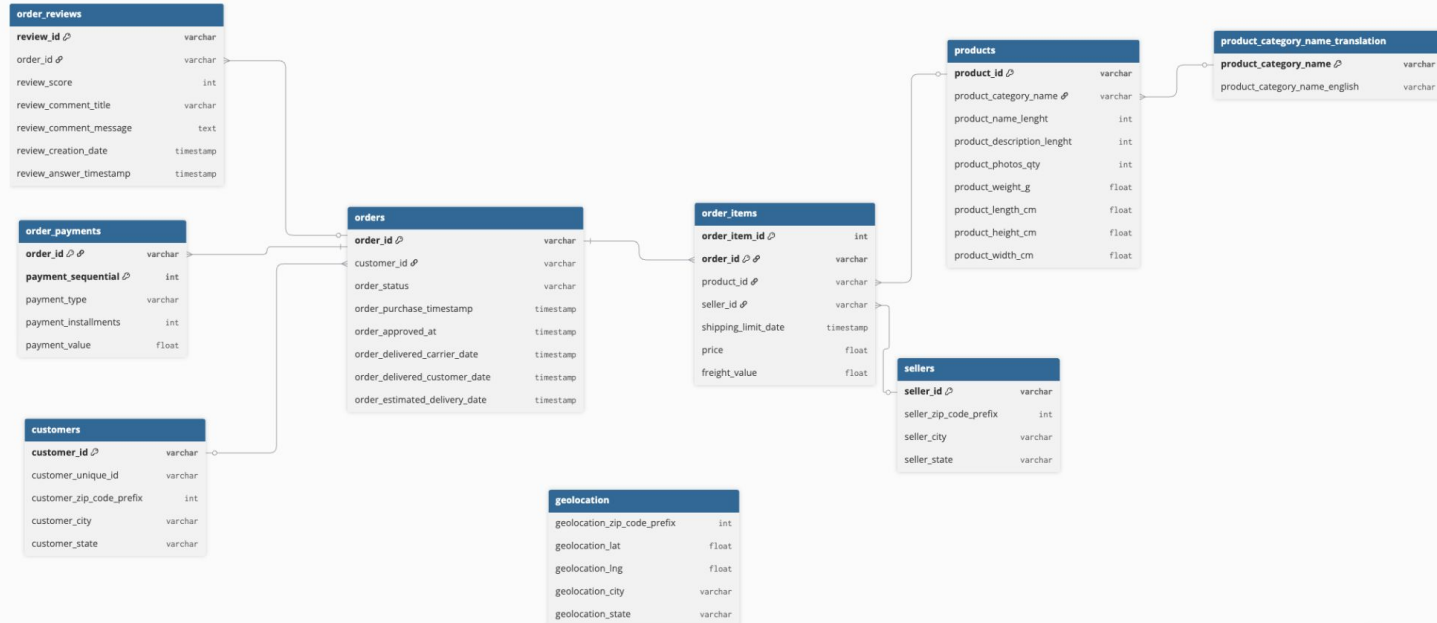




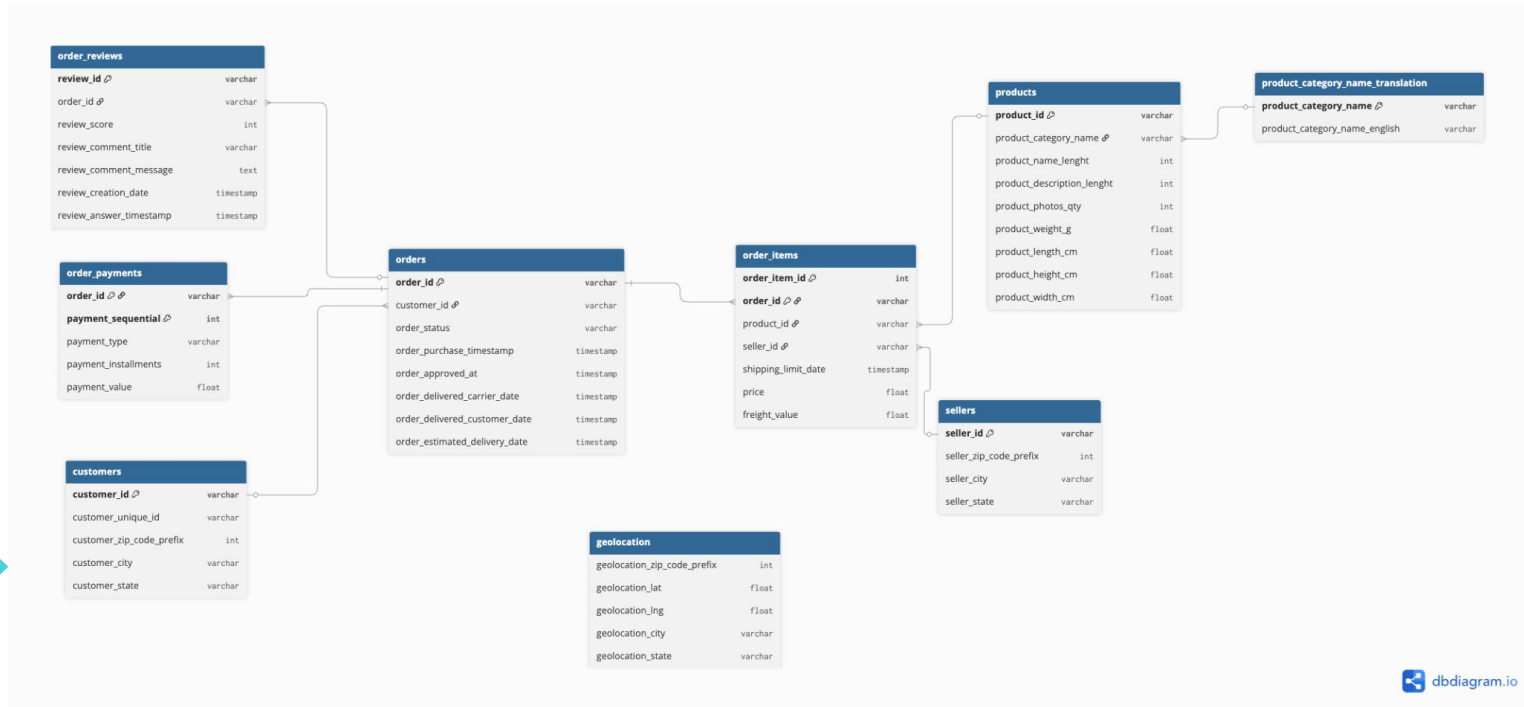
Ejemplo 1

- Vamos a tomar los datos de [este ecommerce](#)
- Veremos una consulta en DB transaccional
- Luego transformaremos los datos
- Consultaremos lo mismo desde BigQuery
- Compararemos

Primero: explorando los datos



Queremos el **total de unidades** y el **ingreso total** por mes y categoría



Queremos el **total de unidades y el ingreso total por mes y categoría**



```
SELECT
    t.product_category_name_english AS category_name,
    EXTRACT(YEAR FROM o.order_purchase_timestamp::DATE) AS sales_year,
    EXTRACT(MONTH FROM o.order_purchase_timestamp::DATE) AS sales_month,
    COUNT(oi.product_id) AS total_units,
    SUM(oi.price) AS total_revenue
FROM orders AS o
JOIN order_items AS oi
    ON o.order_id = oi.order_id
JOIN products AS p
    ON oi.product_id = p.product_id
JOIN product_category_name_translation AS t
    ON p.product_category_name = t.product_category_name
GROUP BY
    category_name,
    sales_year,
    sales_month
ORDER BY
    sales_year DESC,
    sales_month DESC;
```

Análisis con psql

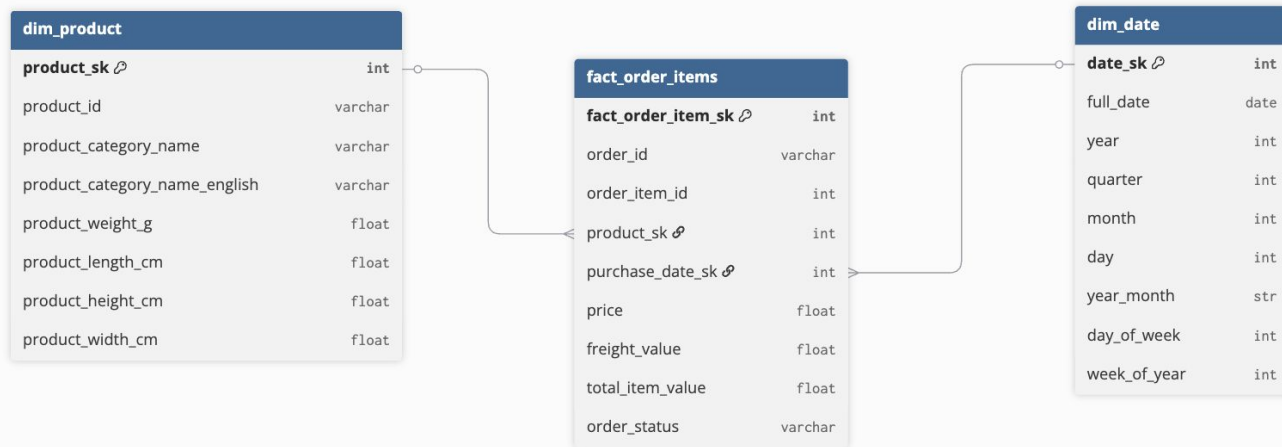
- Tenemos que hacer 3 joins
- Al correr con **EXPLAIN (ANALYZE, BUFFERS)** vemos que la consulta tuvo que escribir en disco para poder completarse:
Sort Method: external merge Disk: 4312kB
- Estamos trayendo de disco muchas columnas que no necesitamos

Cómo lo haríamos si seguimos lo que hemos aprendido hasta ahora...



ETL + BigQuery

- Modelaremos los datos a un esquema menos normalizado



ETL + BigQuery

- Modelaremos los datos a un esquema menos normalizado
- Cargamos a BigQuery

ETL + BigQuery

- Modelaremos los datos a un esquema menos normalizado
- Cargamos a BigQuery
- Consultamos

ETL + BigQuery

- Consultamos

```
SELECT
  p.product_category_name_english AS category,
  d.year_month AS sales_month,
  COUNT(*) AS total_units,
  SUM(f.total_item_value) AS total_revenue
FROM fact_order_items AS f
INNER JOIN dim_product AS p
  ON f.product_sk = p.product_sk
INNER JOIN dim_date AS d
  ON f.purchase_date_sk = d.date_sk
GROUP BY category, sales_month
ORDER BY sales_month, category
```



Ejemplo 2

- Queremos los top 3 productos por mes del año 2017

Ejemplo 2

- Queremos los top 3 productos por mes del año 2017
- Esto no es tan fácil en psql

Ejemplo 2

- Queremos los top 3 productos por mes del año 2017
- Esto no es tan fácil en psql
- Usaremos el mismo modelo de antes

Ejemplo 2

- Queremos los top 3 productos por mes del año 2017

Ejemplo 2

- Queremos los top 3 productos por mes del año 2017
- 1° Recuperamos los datos que nos interesan

```
WITH monthly_product_sales AS (  
  SELECT  
    p.product_id,  
    p.product_category_name_english AS category,  
    d.month AS sales_month,  
    COUNT(f.fact_order_item_sk) AS total_units  
  FROM fact_order_items AS f  
  INNER JOIN dim_product AS p  
    ON f.product_sk = p.product_sk  
  INNER JOIN dim_date AS d  
    ON f.purchase_date_sk = d.date_sk  
  WHERE d.year = 2017  
  GROUP BY 1, 2, 3  
)
```


Ejemplo 2

- Queremos los top 3 productos por mes del año 2017
- 2° Armamos lo que queremos recuperar

SELECT

**sales_month,
product_id,
category,
total_units,**

**** ranking de productos ****

AS product_rank

**FROM monthly_product_sales
QUALIFY product_rank <= 3;**



Ejemplo 2

- Queremos los top 3 productos por mes del año 2017
- 2° Calculamos el ranking por mes y retornamos información

```
SELECT
  sales_month,
  product_id,
  category,
  total_units,
  DENSE_RANK() OVER (
    PARTITION BY sales_month
    ORDER BY total_units DESC
  ) AS product_rank
FROM monthly_product_sales
QUALIFY product_rank <= 3;
```



Ejercicios





Ejercicios

* Ir a GitHub



Lectura complementaria

- Recomiendo mucho leer esta:
<https://count.co/sql-resources/bigquery-standard-sql/window-functions-explained>
- <https://alanezz.notion.site/Clase-03-Window-Functions-8300d89289f742ba81793445694dfed4>
- Datos originales:
<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>